

JavaScript - Day 6

DOM (Document Object Model)

Student Notes & Real-World Project Reference Guide

Topics covered:

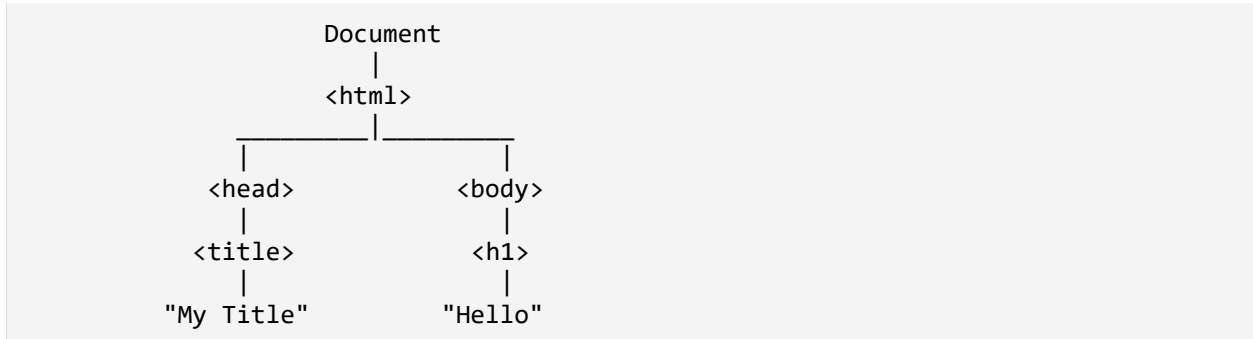
DOM Tree | Selecting Elements | Updating Content & Attributes
Changing CSS | Creating, Adding, Replacing & Removing Elements
Class Manipulation | Useful DOM Properties | Real-World Uses

What is DOM?

Definition

The Document Object Model (DOM) is a programming interface for web pages. It represents an HTML document as a tree of objects that JavaScript can access and manipulate.

DOM Tree Structure



Every HTML tag becomes an object inside this tree.

What Can JavaScript Do Using DOM?

- ✓ Change HTML content
- ✓ Change HTML attributes
- ✓ Change CSS styles
- ✓ Create new elements
- ✓ Remove elements
- ✓ Replace elements
- ✓ Respond to user actions
- ✓ Create dynamic web pages

Selecting HTML Elements

Before modifying an element, JavaScript must first select it.

1. `getElementById()`

Selects an element using its id.

HTML

```
<h1 id="title">JavaScript Session 6</h1>
```

JavaScript

```
let element = document.getElementById("title");  
  
console.log(element);
```

2. getElementsByClassName()

Selects all elements with a given class.

HTML

```
<h4 class="students">Sumanth</h4>  
<h4 class="students">Hemanth</h4>
```

JavaScript

```
let students = document.getElementsByClassName("students");  
  
console.log(students);
```

Returns an HTML Collection.

3. getElementsByTagName()

Selects elements based on tag name.

```
let headings = document.getElementsByTagName("h4");  
  
console.log(headings);
```

Modern Selection Methods

Modern websites commonly use Query Selectors.

4. querySelector()

Returns the first matching element.

HTML

```
<h4 class="students">Sumanth</h4>  
<h4 class="students">Hemanth</h4>
```

JavaScript

```
let student = document.querySelector(".students");
```

```
console.log(student);
```

Only the first element is selected.

5. `querySelectorAll()`

Returns all matching elements.

```
let students = document.querySelectorAll(".students");

console.log(students);
```

Returns a `NodeList`.

Selection Methods Summary

Method	Returns	Selects
<code>getElementById()</code>	Single element	By id
<code>getElementsByClassName()</code>	HTML Collection	By class name
<code>getElementsByTagName()</code>	HTML Collection	By tag name
<code>querySelector()</code>	Single element (first match)	Any CSS selector
<code>querySelectorAll()</code>	<code>NodeList</code>	Any CSS selector (all matches)

Updating HTML Elements

Once an element is selected, we can modify it.

1. `innerHTML`

Updates HTML content. HTML tags inside the string are interpreted.

HTML

```
<h1 id="heading">Welcome</h1>
```

JavaScript

```
document.getElementById("heading").innerHTML =
"<i>Hello Everyone</i>";
```

Output:

```
<i>Hello Everyone</i> (rendered in italics)
```

The text becomes italic because HTML tags are interpreted.

2. innerText

Updates only visible text.

```
element.innerText = "Hello Students";
```

3. textContent

Updates all text, including hidden text.

```
element.textContent = "Updated Text";
```

innerText vs textContent

innerText	textContent
Considers visible text only	Considers all text
Affected by CSS	Not affected by CSS
Slightly slower	Faster

Working with Attributes

HTML attributes can be accessed and modified.

getAttribute()

Used to read attribute values.

```
let image = document.getElementById("image");  
  
console.log(image.getAttribute("src"));
```

setAttribute()

Used to update attributes.

```
image.setAttribute("src", "../images/2.jpg");  
  
image.setAttribute("height", "300px");
```

Changing CSS Using JavaScript

Styles can be modified using the style property.

HTML

```
<h1 id="heading">Welcome</h1>
```

JavaScript

```
let heading = document.getElementById("heading");

heading.style.color = "red";

heading.style.fontSize = "40px";
```

CSS Property Conversion

CSS uses hyphens (-). JavaScript uses camelCase.

CSS	JavaScript
font-size	fontSize
background-color	backgroundColor
padding-left	paddingLeft
margin-top	marginTop

Inserting Elements Before Another Element

insertBefore()

Adds an element before a specified sibling element.

HTML

```
<ul id="list">
  <li>Apple</li>
  <li id="banana">Banana</li>
</ul>
```

JavaScript

```
let item = document.createElement("li");

item.innerText = "Mango";

let banana = document.getElementById("banana");

list.insertBefore(item, banana);
```

Output:

```
Apple  
Mango  
Banana
```

Creating Elements

Sometimes elements don't exist in HTML. JavaScript can create them dynamically.

createElement()

Creates a new HTML element.

```
let heading = document.createElement("h1");
```

createTextNode()

Creates text content.

```
let text = document.createTextNode(  
  "I created using JavaScript"  
);
```

Combining Both

```
let heading = document.createElement("h1");  
  
let text = document.createTextNode(  
  "I created using JavaScript"  
);  
  
heading.appendChild(text);
```

Adding Elements

After creating an element, it must be added to the page.

appendChild()

Adds an element as the last child.

```
div.appendChild(heading);
```

Example

```
let heading = document.createElement("h1");
```

```
heading.innerText = "Hello Students";

document.body.appendChild(heading);
```

append()

Can add text and elements.

```
div.append(" and Hello Students");
```

prepend()

Adds content at the beginning.

```
div.prepend("Starting and ");
```

appendChild() vs append()

appendChild()	append()
Adds only nodes	Adds text and nodes
Older method	Modern method

Replacing Elements

Sometimes existing elements must be replaced.

Old Method: replaceChild()

Called on the parent element.

```
parent.replaceChild(
  newContent,
  oldContent
);
```

Example

```
let newContent = document.createElement("div");

newContent.innerText = "New Content";

parent.replaceChild(
  newContent,
  oldContent
);
```


Modern Method: `replaceWith()`

```
oldContent.replaceWith(newContent);
```

Simpler and more readable.

Removing Elements

Elements can also be deleted.

Old Method: `removeChild()`

Called on the parent.

```
parent.removeChild(child);
```

Example

```
let parent =  
document.getElementById("list");  
  
let child =  
document.getElementById("banana");  
  
parent.removeChild(child);
```

Modern Method: `remove()`

Directly removes an element.

```
document  
  .getElementById("dragonFruit")  
  .remove();
```

Class Manipulation

JavaScript can add, remove, or toggle CSS classes.

`classList` Property

Contains all classes of an element.

```
console.log(element.classList);
```

1. `classList.add()`

Adds a class.

```
list.classList.add("big");
```

2. `classList.remove()`

Removes a class.

```
list.classList.remove("big");
```

3. `classList.toggle()`

Adds the class if missing. Removes the class if present.

```
list.classList.toggle("big");
```

Example CSS

```
.big{
  font-size:30px;
  color:green;
}

.small{
  font-size:16px;
  color:red;
}
```

Example Usage

```
list.classList.remove("big");

list.classList.add("small");
```

Output changes based on the applied class.

Useful DOM Properties

`childElementCount`

Returns the number of child elements.

```
console.log(
  list.childElementCount
);
```

Output:

```
4
```

firstElementChild

Returns the first child element.

```
console.log(
  list.firstElementChild
);
```

cloneNode()

Creates a copy of an element.

```
let copy = element.cloneNode(true);
```

true

Copies the element and all its children.

```
element.cloneNode(true);
```

false

Copies only the element (no children).

```
element.cloneNode(false);
```

Real-World Uses of DOM

E-Commerce Website

- Add products dynamically
- Update cart count
- Remove products from cart

Social Media

- Add comments
- Delete comments
- Update likes

Online Forms

- Validate input fields
- Show error messages
- Add dynamic fields

Dashboard Applications

- Update data without reloading
- Create charts dynamically

Common Beginner Mistakes

Forgetting to Select Element

✗ Wrong

```
heading.innerText = "Hello";
```

✓ Correct

```
let heading =  
document.getElementById("heading");  
  
heading.innerText = "Hello";
```

Using innerHTML for Plain Text

✗ Wrong

```
element.innerHTML = "Hello";
```

✓ Better

```
element.innerText = "Hello";
```

Note: Use `innerText/textContent` for plain text. Reserve `innerHTML` for inserting actual HTML markup — and be cautious, since it can introduce security risks with untrusted input.

Calling `appendChild` on null

✗ Risky

```
document  
  .getElementById("missing")  
  .appendChild(element);
```

Always verify the element exists before calling methods on it.

Using CSS Names Directly

✗ Wrong

```
heading.style.font-size = "30px";
```

✓ Correct

```
heading.style.fontSize = "30px";
```

Practice Exercises

Exercise 1

Create an `<h2>` element using JavaScript and display:

```
Welcome to DOM
```

Exercise 2

Change the text of a paragraph using:

- `innerText`
- `textContent`

Exercise 3

Change the image source when a button is clicked.

Exercise 4

Create a new `<li>` element and add it to a list.

Exercise 5

Remove a specific item from a list.

Exercise 6

Replace a heading with another heading.

Exercise 7

Create a button that toggles a class named:

```
dark-mode
```

Exercise 8

Display the number of child elements inside a list.

Mini Assignment

Create a Student List Manager using DOM.

Requirements:

1. Add student names dynamically.

2. Delete student names.
3. Replace a student name.
4. Toggle list styles using classes.
5. Display total number of students.

Key Takeaways

- ✓ DOM represents HTML as JavaScript objects.
- ✓ JavaScript uses DOM to access and manipulate web pages.
- ✓ Elements can be selected using `getElementById()`, `getElementsByClassName()`, `getElementsByTagName()`, `querySelector()`, and `querySelectorAll()`.
- ✓ Content can be updated using `innerHTML`, `innerText`, and `textContent`.
- ✓ Attributes can be modified using `getAttribute()` and `setAttribute()`.
- ✓ Elements can be created, added, replaced, and removed.
- ✓ Classes can be managed using `classList.add()`, `classList.remove()`, and `classList.toggle()`.
- ✓ DOM is the foundation of dynamic and interactive web applications.